TRAVAUX PRATIQUES

CI/CD avec GitHub Actions

Dashboard Météo Automatique

Construisez un pipeline CI/CD complet qui récupère automatiquement les données météo en temps réel et met à jour votre README GitHub.

GitHub Actions	API Météo	Python 3	YAML

Niveau : **Débutant** | Durée : **2-3 heures**

Outils : Git, Python, GitHub, VS Code

Sommaire

1 Introduction au CI/CD

Introduction au CI/CD

Qu'est-ce que le CI/CD ?

Le **CI/CD** (Continuous Integration / Continuous Deployment) est un ensemble de pratiques qui permettent d'automatiser les étapes de développement logiciel. L'idée est simple : à chaque modification de code, des actions automatiques se déclenchent pour tester, construire et déployer votre application.

> ℹ️ **INFO**
> CI (Intégration Continue) : À chaque push, le code est automatiquement testé et validé. Cela permet de détecter les erreurs rapidement avant qu'elles n'atteignent la production.

> ✅ **ASTUCE**
> CD (Déploiement Continu) : Une fois validé, le code est automatiquement déployé en production ou met à jour des ressources (comme notre README dans ce TP).

Pourquoi GitHub Actions ?

GitHub Actions est l'outil de CI/CD intégré directement dans GitHub. Il permet de créer des **workflows** (pipelines) qui s'exécutent automatiquement en réponse à des événements (push, pull request, planification cron, etc.). C'est **gratuit** pour les repositories publics et très facile à prendre en main pour les débutants.

Notre projet : Dashboard Météo dans le README

Dans ce TP, vous allez créer un pipeline CI/CD qui :

- Se déclenche automatiquement toutes les 6 heures (et manuellement)
- Exécute un script Python qui appelle une API météo en temps réel
- Formate les données météo (température, humidité, vent, conditions)
- Met à jour automatiquement le fichier README.md du repository
- Fait un commit et un push automatique des modifications

> ✅ **ASTUCE**
> C'est un excellent projet pour débutants ! Il est concret (on voit le résultat dans le README), couvre les concepts clés du CI/CD, et utilise des technologies réelles (API REST, Python, YAML, Git). Le fait de voir son README se mettre à jour tout seul est très motivant !

Architecture du projet

Voici le flux de données de notre pipeline CI/CD :

Déclencheur	Orchestrateur	Traitement	Résultat
Cron (6h) ou Manuel	GitHub Actions	Script Python	README.md mis à jour
workflow_dispatch	Runner Ubuntu	API OpenWeatherMap	Commit + Push auto

2 Prérequis et Installation

Prérequis et Installation

Ce dont vous avez besoin

1. **Un compte GitHub**
   Gratuit sur github.com — créez-en un si vous n'en avez pas encore.

2. **Git installé**
   Téléchargez-le sur git-scm.com ou installez-le via votre gestionnaire de paquets.

3. **Python 3.8+**
   Téléchargez-le sur python.org. Vérifiez avec : python --version

4. **Un éditeur de code**
   VS Code est recommandé (gratuit), mais tout éditeur de texte fonctionne.

5. **Une connexion Internet**
   Nécessaire pour l'API météo et GitHub.

Vérifier les installations

Ouvrez un terminal et tapez les commandes suivantes pour vérifier que tout est en place :

Terminal — Vérification bash

```bash
# Vérifier Git
git --version
# Attendu : git version 2.x.x

# Vérifier Python
python3 --version
# Attendu : Python 3.8+

# Vérifier pip
pip3 --version
# Attendu : pip 2x.x
```

> ⚠️ **ATTENTION**
> Si python3 ne fonctionne pas, essayez python (sans le 3). Sur Windows, vous devrez peut-être utiliser py au lieu de python3.

3 Créer le Repository GitHub

Étape 1 — Créer le Repository GitHub

3.1 — Créer un nouveau repository

1 **Aller sur github.com**
Connectez-vous à votre compte GitHub.

2 **Cliquer sur « New repository »**
Bouton vert « + » en haut à droite, puis « New repository ».

3 **Configurer le repository**
Nom : weather-dashboard-cicd | Public | Cochez « Add a README file »

4 **Cliquer sur « Create repository »**
Votre repo est prêt !

3.2 — Cloner le repository en local

Ouvrez votre terminal et clonez le repository sur votre machine :

Terminal — Cloner le repo bash

```bash
# Cloner le repository (remplacez VOTRE-USERNAME)
git clone https://github.com/VOTRE-USERNAME/weather-dashboard-cicd.git

# Entrer dans le dossier
cd weather-dashboard-cicd

# Vérifier que tout est ok
git status
```

3.3 — Structure du projet

Voici la structure finale de fichiers que nous allons créer au fil de ce TP :

Structure du projet

```
weather-dashboard-cicd/
├── .github/
│   └── workflows/
│       └── update_weather.yml    # Pipeline CI/CD
├── update_weather.py             # Script Python
├── requirements.txt              # Dépendances Python
```

```
└── README.md                    # Fichier mis à jour automatiquement
```

4 Obtenir une Clé API Météo

Étape 2 — Obtenir une Clé API Météo

Nous allons utiliser **OpenWeatherMap**, une API météo gratuite et fiable. Le plan gratuit permet **1000 appels par jour**, largement suffisant pour notre projet.

1 **Créer un compte**
Allez sur openweathermap.org/api et cliquez sur « Sign Up ».

2 **Confirmer votre email**
Vérifiez votre boîte mail et cliquez sur le lien de confirmation.

3 **Récupérer votre clé API**
Allez dans votre profil > « API keys ». Copiez la clé par défaut.

4 **Attendre l'activation**
La clé peut prendre 10-30 minutes pour s'activer. Patience !

Tester votre clé API

Pour vérifier que votre clé fonctionne, utilisez cette commande dans votre terminal :

Terminal — Tester l'API bash

```bash
# Remplacez VOTRE_CLE_API par votre clé
curl "https://api.openweathermap.org/data/2.5/weather?
q=Paris&appid=VOTRE_CLE_API&units=metric&lang=fr"

# Réponse attendue (extrait) :
# {"main":{"temp":15.2,"humidity":72},
#  "weather":[{"description":"nuageux"}]}
```

IMPORTANT
Ne JAMAIS mettre votre clé API directement dans le code source ! Nous utiliserons les « Secrets » de GitHub pour la stocker de manière sécurisée. C'est une bonne pratique fondamentale en CI/CD et en développement.

5 Créer le Script Python

Étape 3 — Créer le Script Python

5.1 — Créer le fichier requirements.txt

Ce fichier liste les dépendances Python nécessaires. Créez-le à la racine du projet :

requirements.txt

```
requests==2.31.0
```

5.2 — Créer le script update_weather.py

C'est le cœur de notre projet. Ce script va : appeler l'API météo, formater les données, et générer le contenu du README.md. Créez ce fichier à la racine du projet.

> 📝 **NOTE**
> Lisez attentivement chaque commentaire dans le code ! Ils expliquent ce que fait chaque partie.

update_weather.py — Partie 1/4 : Configuration python

```python
import requests
import os
from datetime import datetime

# ============================================
# CONFIGURATION
# ============================================
# La clé API est récupérée depuis les variables
# d'environnement (jamais en dur dans le code !)
API_KEY = os.environ.get("OPENWEATHER_API_KEY")

# Liste des villes à afficher
# Modifiez cette liste selon vos préférences !
CITIES = ["Paris", "London", "New York", "Tokyo", "Sydney"]

# URL de base de l'API OpenWeatherMap
BASE_URL = "https://api.openweathermap.org/data/2.5/weather"
```

update_weather.py — Partie 2/4 : Fonction API python

```python
# ============================================
# FONCTION : Récupérer la météo d'une ville
# ============================================
def get_weather(city):
    """Appelle l'API et retourne les données météo."""
    params = {
```

```
        "q": city,           # Nom de la ville
        "appid": API_KEY,    # Clé API
        "units": "metric",   # Températures en Celsius
        "lang": "fr"         # Descriptions en français
    }
    try:
        response = requests.get(BASE_URL, params=params)
        response.raise_for_status()  # Lève une erreur si status != 200
        data = response.json()
        return {
            "city": city,
            "temp": round(data["main"]["temp"], 1),
            "feels_like": round(data["main"]["feels_like"], 1),
            "humidity": data["main"]["humidity"],
            "description": data["weather"][0]["description"],
            "wind": round(data["wind"]["speed"] * 3.6, 1),
            "icon": get_weather_emoji(data["weather"][0]["main"])
        }
    except Exception as e:
        print(f"Erreur pour {city}: {e}")
        return None
```

update_weather.py — Partie 3/4 : Emojis + Générateur README python

```python
# ============================================
# FONCTION : Emoji selon la condition météo
# ============================================
def get_weather_emoji(condition):
    """Retourne un emoji selon la condition météo."""
    emojis = {
        "Clear": "☀️",
        "Clouds": "☁️",
        "Rain": "🌧️",
        "Drizzle": "🌦️",
        "Thunderstorm": "⛈️",
        "Snow": "❄️",
        "Mist": "🌫️",
        "Fog": "🌫️",
        "Haze": "🌫️",
    }
    return emojis.get(condition, "🌡️")


# ============================================
# FONCTION : Générer le contenu du README
# ============================================
def generate_readme(weather_data):
    """Crée le contenu Markdown du README."""
    now = datetime.utcnow().strftime("%d/%m/%Y à %H:%M UTC")

    readme = f"""# 🌤️ Dashboard Météo - CI/CD

> Ce README est mis à jour automatiquement
> par GitHub Actions toutes les 6 heures !

## Meteo actuelle - {now}

| Ville | Météo | Temp | Ressenti | Humidité | Vent |
|-------|-------|------|----------|----------|------|
"""
    for w in weather_data:
        if w:
            readme += (
                f"| {w['icon']} {w['city']} "
                f"| {w['description'].capitalize()} "
                f"| {w['temp']}°C "
                f"| {w['feels_like']}°C "
                f"| {w['humidity']}% "
                f"| {w['wind']} km/h |\n"
            )
    return readme
```

update_weather.py — Partie 4/4 : Exécution python

```python
# ============================================
# POINT D'ENTRÉE DU SCRIPT
# ============================================
if __name__ == "__main__":
    # Vérifier que la clé API est définie
```

```
    if not API_KEY:
        print("ERREUR : OPENWEATHER_API_KEY non définie !")
        exit(1)

    # Récupérer la météo de chaque ville
    print("Récupération des données météo...")
    weather_data = []
    for city in CITIES:
        data = get_weather(city)
        weather_data.append(data)
        if data:
            print(f"  {city}: {data['temp']}°C")

    # Générer et écrire le README
    print("Génération du README...")
    readme_content = generate_readme(weather_data)

    with open("README.md", "w", encoding="utf-8") as f:
        f.write(readme_content)

    print("README.md mis à jour avec succès !")
```

> ℹ️ **INFO**
> os.environ.get('OPENWEATHER_API_KEY') récupère la clé API depuis les variables d'environnement. C'est GitHub Actions qui injectera cette variable à partir des secrets. Ainsi, la clé n'apparaît jamais dans le code.

6 Créer le Workflow GitHub Actions

Étape 4 — Créer le Workflow GitHub Actions

C'est la partie la plus importante du TP ! Le fichier workflow définit notre pipeline CI/CD. Il doit être placé dans le dossier **.github/workflows/**.

6.1 — Créer la structure de dossiers

Terminal bash

```bash
# Créer les dossiers nécessaires
mkdir -p .github/workflows
```

6.2 — Créer le fichier workflow

Créez le fichier **.github/workflows/update_weather.yml** avec le contenu suivant. Chaque section est commentée pour vous aider à comprendre :

update_weather.yml — Partie 1/3 : Déclencheurs yaml

```yaml
# Nom du workflow (affiché dans l'onglet Actions)
name: Update Weather Dashboard

# ============================================
# DÉCLENCHEURS : quand le workflow s'exécute
# ============================================
on:
  schedule:
    # Planification cron : toutes les 6 heures
    # Format : Minute Heure Jour Mois JourSemaine
    - cron: '0 */6 * * *'

  # Permet le lancement manuel depuis GitHub
  workflow_dispatch:

  # Se lance aussi à chaque push sur main
  push:
    branches: [ main ]
```

update_weather.yml — Partie 2/3 : Job + Setup yaml

```yaml
# ============================================
# JOBS : les tâches à exécuter
# ============================================
jobs:
  update-weather:
    # Machine virtuelle utilisée par GitHub
    runs-on: ubuntu-latest
```

```
    # Permissions pour pouvoir push sur le repo
    permissions:
      contents: write

    # Étapes séquentielles du job
    steps:
      # Étape 1 : Récupérer le code du repo
      - name: Checkout du repository
        uses: actions/checkout@v4

      # Étape 2 : Installer Python
      - name: Installer Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
```

update_weather.yml — Partie 3/3 : Build + Deploy yaml

```yaml
      # Étape 3 : Installer les dépendances Python
      - name: Installer les dépendances
        run: pip install -r requirements.txt

      # Étape 4 : Exécuter le script météo
      # Le secret est injecté comme variable d'env
      - name: Mettre à jour la météo
        env:
          OPENWEATHER_API_KEY: ${{ secrets.OPENWEATHER_API_KEY }}
        run: python update_weather.py

      # Étape 5 : Commit et push des changements
      - name: Commit et push
        run: |
          git config --local user.email "action@github.com"
          git config --local user.name "GitHub Action Bot"
          git add README.md
          git diff --staged --quiet || (
            git commit -m "MAJ météo auto - $(date -u)"
            git push
          )
```

6.3 — Comprendre le workflow YAML

Voici un tableau récapitulatif des mots-clés importants du fichier YAML :

Mot-clé	Rôle
name:	Nom affiché dans l'onglet Actions de GitHub
on:	Événements qui déclenchent le workflow (cron, push, manuel)
jobs:	Les tâches à exécuter. Chaque job tourne sur une VM isolée
runs-on:	Type de machine virtuelle (ubuntu-latest = Ubuntu dernière version)
steps:	Étapes séquentielles du job
uses:	Utilise une Action précréée (ex: actions/checkout@v4)
run:	Exécute une commande shell directement
env:	Variables d'environnement injectées (ici, le secret API)
secrets.X:	Accède à un secret stocké dans les paramètres du repo

7 Configurer les Secrets GitHub

Étape 5 — Configurer les Secrets GitHub

Les **Secrets GitHub** permettent de stocker des informations sensibles (clés API, mots de passe, tokens) de manière **sécurisée**. Ils sont chiffrés et ne sont jamais affichés dans les logs d'exécution.

1 **Ouvrir votre repository**
Allez sur github.com/VOTRE-USERNAME/weather-dashboard-cicd

2 **Aller dans Settings**
Cliquez sur l'onglet « Settings » du repository (pas vos settings personnels).

3 **Secrets and variables > Actions**
Dans le menu de gauche, sous la section « Security ».

4 **New repository secret**
Cliquez sur le bouton vert « New repository secret ».

5 **Configurer le secret**
Name : OPENWEATHER_API_KEY | Value : collez votre clé API

6 **Cliquer sur Add secret**
Le secret est maintenant stocké en sécurité !

IMPORTANT
Le nom du secret doit correspondre EXACTEMENT à celui utilisé dans le workflow YAML : OPENWEATHER_API_KEY. Les secrets sont accessibles via ${{ secrets.NOM_DU_SECRET }} dans le fichier YAML.

8 Tester et Valider le Pipeline

Étape 6 — Tester et Valider le Pipeline

8.1 — Pousser le code sur GitHub

Maintenant que tous les fichiers sont créés, il est temps de pousser votre code :

Terminal — Premier push bash

```bash
# Ajouter tous les fichiers
git add .

# Vérifier ce qui sera commité
git status

# Commiter les changements
git commit -m "feat: ajout du pipeline CI/CD météo"

# Pousser sur GitHub
git push origin main
```

8.2 — Vérifier l'exécution du workflow

1 **Aller dans l'onglet Actions**
Sur votre repo GitHub, cliquez sur « Actions » dans la barre de navigation.

2 **Observer l'exécution**
Vous devriez voir un workflow en cours (point jaune) ou terminé (coche verte).

3 **Cliquer sur le workflow**
Vous pouvez voir le détail de chaque étape et les logs d'exécution.

4 **Vérifier le README**
Retournez sur la page principale du repo : le README doit être mis à jour avec la météo !

8.3 — Lancement manuel (workflow_dispatch)

Vous pouvez aussi déclencher le workflow manuellement à tout moment :

1. Allez dans l'onglet « Actions » de votre repository
2. Cliquez sur « Update Weather Dashboard » dans la liste de gauche
3. Cliquez sur le bouton « Run workflow » en haut à droite
4. Sélectionnez la branche « main » et confirmez

8.4 — Dépannage

Si quelque chose ne fonctionne pas, consultez ce tableau :

Problème	Cause probable	Solution
Workflow ne se lance pas	Fichier YAML mal placé	Vérifier : .github/workflows/
Erreur API key	Secret mal configuré	Vérifier le nom exact du secret
Erreur Python	Dépendance manquante	Vérifier requirements.txt
Push échoue	Permissions insuffisantes	Ajouter permissions: contents: write
README non mis à jour	Pas de changement détecté	Normal si météo identique

9 Pour Aller Plus Loin

Pour Aller Plus Loin — Améliorations

Maintenant que votre pipeline fonctionne, voici des pistes d'amélioration classées par difficulté :

Niveau 1 — Améliorations simples

- ☐ Ajouter plus de villes (votre ville, villes préférées...)
- ☐ Personnaliser les emojis et le format du tableau Markdown
- ☐ Changer la fréquence de mise à jour (toutes les heures, etc.)
- ☐ Ajouter la date de lever/coucher du soleil

Niveau 2 — Améliorations intermédiaires

- ☐ Ajouter des prévisions sur 5 jours (endpoint /forecast)
- ☐ Générer un graphique SVG des températures avec matplotlib
- ☐ Ajouter un badge de statut du workflow dans le README
- ☐ Créer un workflow de test qui vérifie le script avant le déploiement

Niveau 3 — Améliorations avancées

- ☐ Déployer un site HTML sur GitHub Pages avec les données météo
- ☐ Ajouter des tests unitaires avec pytest et un job de test séparé
- ☐ Créer une matrice de jobs (tester sur Python 3.9, 3.10, 3.11, 3.12)
- ☐ Ajouter des notifications Slack ou Discord en cas d'échec du pipeline
- ☐ Mettre en place un système de cache pour éviter les appels API inutiles

Exemple bonus : Pipeline multi-jobs

Pour aller plus loin, vous pouvez séparer les tests et le déploiement en deux jobs distincts. Le job « deploy » ne s'exécute que si le job « test » réussit :

Pipeline multi-jobs (avancé) yaml

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
```

```
      - run: pip install -r requirements.txt
      - run: python -m pytest tests/ -v

  deploy:
    needs: test    # Attend que 'test' réussisse !
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      # ... étapes de déploiement
```

10 Annexes et Références

Annexes et Références

Glossaire CI/CD

Terme	Définition
Pipeline	Ensemble d'étapes automatisées exécutées séquentiellement
Workflow	Fichier YAML qui définit un pipeline dans GitHub Actions
Job	Groupe d'étapes qui s'exécute sur une même machine virtuelle
Step	Une action individuelle dans un job (commande ou action)
Runner	Machine virtuelle qui exécute les jobs (hébergée par GitHub)
Secret	Variable sensible stockée de façon chiffrée dans GitHub
Cron	Syntaxe pour planifier des tâches récurrentes
Action	Module réutilisable (ex: actions/checkout, actions/setup-python)
Artifact	Fichier généré par un workflow et conservé pour téléchargement

Aide-mémoire Cron

La syntaxe cron définit quand un workflow doit s'exécuter automatiquement. Le format est : **minute heure jour mois jour_semaine**

Expression cron	Fréquence
0 */6 * * *	Toutes les 6 heures
0 * * * *	Toutes les heures
0 0 * * *	Tous les jours à minuit UTC
0 9 * * 1-5	Du lundi au vendredi à 9h UTC
30 8 * * 1	Chaque lundi à 8h30 UTC
0 0 1 * *	Le 1er de chaque mois

✅ **ASTUCE**

> Utilisez le site crontab.guru pour créer et visualiser vos expressions cron de manière interactive. C'est un outil très pratique !

Ressources utiles

- Documentation GitHub Actions : docs.github.com/en/actions
- API OpenWeatherMap : openweathermap.org/api
- Générateur cron interactif : crontab.guru
- Spécification YAML : yaml.org/spec
- Git pour les débutants : git-scm.com/book/fr

✅ Félicitations ! Vous avez créé votre premier pipeline CI/CD fonctionnel. Vous maîtrisez maintenant les bases de GitHub Actions : workflows, jobs, steps, secrets, cron, et l'automatisation. Continuez à expérimenter !

Checklist de validation du TP

Cochez chaque élément une fois terminé :

- ☐ Repository GitHub créé et cloné en local
- ☐ Clé API OpenWeatherMap obtenue et testée
- ☐ Script update_weather.py fonctionnel
- ☐ Fichier requirements.txt créé
- ☐ Workflow .github/workflows/update_weather.yml créé
- ☐ Secret OPENWEATHER_API_KEY configuré dans GitHub
- ☐ Premier push effectué avec succès
- ☐ Workflow exécuté avec succès (coche verte dans Actions)
- ☐ README.md mis à jour automatiquement avec les données météo